

Design Principles and Patterns

Exercise 1: Implementing the Singleton Pattern

Program :

```
public class SingletonPatternExample {  
  
    static class Logger {  
        private static Logger instance;  
  
        private Logger() {  
            System.out.println("Logger instance created.");  
        }  
  
        public static Logger getInstance() {  
            if (instance == null) {  
                instance = new Logger();  
            }  
            return instance;  
        }  
  
        public void log(String message) {  
            System.out.println("LOG: " + message);  
        }  
    }  
  
    public static void main(String[] args) {  
  
        Logger logger1 = Logger.getInstance();  
        Logger logger2 = Logger.getInstance();  
  
        logger1.log("First log message");  
        logger2.log("Second log message");  
  
        if (logger1 == logger2) {  
            System.out.println("Only one Logger instance exists.");  
        } else {  
            System.out.println("Multiple Logger instances exist.");  
        }  
    }  
}
```

Output :

```
Logger instance created.  
LOG: First log message  
LOG: Second log message  
Only one Logger instance exists.
```

Exercise 2: Implementing the Factory Method Pattern

Program :

```

abstract class Document {
    abstract void open();
}

class WordDocument extends Document {
    void open() {
        System.out.println("Opening Word Document");
    }
}

class PdfDocument extends Document {
    void open() {
        System.out.println("Opening PDF Document");
    }
}

class ExcelDocument extends Document {
    void open() {
        System.out.println("Opening Excel Document");
    }
}

abstract class DocumentFactory {
    abstract Document createDocument();
}

class WordFactory extends DocumentFactory {
    Document createDocument() {
        return new WordDocument();
    }
}

class PdfFactory extends DocumentFactory {
    Document createDocument() {
        return new PdfDocument();
    }
}

class ExcelFactory extends DocumentFactory {
    Document createDocument() {
        return new ExcelDocument();
    }
}

public class Main {
    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}

```

```
}
```

Output :

Opening Word Document
Opening PDF Document
Opening Excel Document

Exercise 3: Implementing the Builder Pattern

Program :

```
class Computer {
    private String cpu;
    private int ram;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
    }

    public void show() {
        System.out.println("CPU: " + cpu + ", RAM: " + ram + "GB");
    }

    static class Builder {
        private String cpu;
        private int ram;

        Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }

        Computer build() {
            return new Computer(this);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Computer c1 = new Computer.Builder()
            .setCPU("Intel i7")
            .setRAM(16)
            .build();

        c1.show();
    }
}
```

```
}
```

Output :

CPU: Intel i7, RAM: 16GB

Exercise 4: Implementing the Adapter Pattern

Program :

```
interface PaymentProcessor {
    void processPayment(double amount);
}

class PayPalGateway {
    void sendPayment(double amount) {
        System.out.println("Payment of Rs." + amount + " processed through PayPal.");
    }
}

class StripeGateway {
    void makePayment(double amount) {
        System.out.println("Payment of Rs." + amount + " processed through Stripe.");
    }
}

class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway paypal = new PayPalGateway();

    public void processPayment(double amount) {
        paypal.sendPayment(amount);
    }
}

class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripe = new StripeGateway();

    public void processPayment(double amount) {
        stripe.makePayment(amount);
    }
}

public class Main {
    public static void main(String[] args) {

        PaymentProcessor paypal = new PayPalAdapter();
        paypal.processPayment(1000);

        PaymentProcessor stripe = new StripeAdapter();
        stripe.processPayment(2000);
    }
}
```

Output :

Payment of Rs.1000.0 processed through PayPal.
Payment of Rs.2000.0 processed through Stripe.j

Exercise 5: Implementing the Decorator Pattern

Program :

```
interface Notifier {
    void send(String message);
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

abstract class NotifierDecorator implements Notifier {
    protected Notifier notifier;

    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
    }
}

class SMSNotifierDecorator extends NotifierDecorator {

    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("SMS: " + message);
    }
}

class SlackNotifierDecorator extends NotifierDecorator {

    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("Slack: " + message);
    }
}

public class Main {
    public static void main(String[] args) {
```

```

        Notifier notifier = new SlackNotifierDecorator(
            new SMSNotifierDecorator(
                new EmailNotifier());

        notifier.send("Hello User!");
    }
}

```

Output :

Email: Hello User!
 SMS: Hello User!
 Slack: Hello User!

Exercise 6: Implementing the Proxy Pattern

Program :

```

interface Image {
    void display();
}

class ReallImage implements Image {
    private String fileName;

    public ReallImage(String fileName) {
        this.fileName = fileName;
        loadFromServer();
    }

    private void loadFromServer() {
        System.out.println("Loading " + fileName + " from remote server...");
    }

    public void display() {
        System.out.println("Displaying " + fileName);
    }
}

class ProxyImage implements Image {
    private String fileName;
    private ReallImage reallImage;

    public ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    public void display() {
        if (reallImage == null) {
            reallImage = new ReallImage(fileName);
        }
        reallImage.display();
    }
}

public class Main {

```

```

    public static void main(String[] args) {
        Image image = new ProxyImage("sample.jpg");
        image.display();
        System.out.println();
        image.display();
    }
}

```

Output :

Loading sample.jpg from remote server...
 Displaying sample.jpg

Displaying sample.jpg

Exercise 7: Implementing the Observer Pattern

Program :

```

import java.util.*;

interface Observer {
    void update(int price);
}

class MobileApp implements Observer {
    public void update(int price) {
        System.out.println("Mobile App: " + price);
    }
}

class WebApp implements Observer {
    public void update(int price) {
        System.out.println("Web App: " + price);
    }
}

class StockMarket {
    List<Observer> observers = new ArrayList<>();

    void register(Observer o) {
        observers.add(o);
    }

    void setPrice(int price) {
        for (Observer o : observers)
            o.update(price);
    }
}

public class Main {
    public static void main(String[] args) {
        StockMarket stock = new StockMarket();

        stock.register(new MobileApp());
        stock.register(new WebApp());
    }
}

```

```
        stock.setPrice(100);
    }
}
```

Output :

Mobile App: 100
Web App: 100

Exercise 8: Implementing the Strategy Pattern

Program :

```
interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using PayPal");
    }
}

class PaymentContext {
    private PaymentStrategy strategy;

    PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    void executePayment(int amount) {
        strategy.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {

        PaymentContext p1 =
            new PaymentContext(new CreditCardPayment());
        p1.executePayment(1000);

        PaymentContext p2 =
            new PaymentContext(new PayPalPayment());
        p2.executePayment(2000);
    }
}
```

Output :

Paid Rs.1000 using Credit Card

Exercise 9: Implementing the Command Pattern

Program :

```
interface Command {
    void execute();
}

class Light {
    void on() {
        System.out.println("Light is ON");
    }

    void off() {
        System.out.println("Light is OFF");
    }
}

class LightOnCommand implements Command {
    Light light;

    LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }
}

class LightOffCommand implements Command {
    Light light;

    LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }
}

class RemoteControl {
    Command command;

    void setCommand(Command command) {
        this.command = command;
    }

    void pressButton() {
        command.execute();
    }
}

public class Main {
    public static void main(String[] args) {
```

```

    Light light = new Light();
    RemoteControl remote = new RemoteControl();

    remote.setCommand(new LightOnCommand(light));
    remote.pressButton();

    remote.setCommand(new LightOffCommand(light));
    remote.pressButton();
}
}

```

Output :

Light is ON
Light is OFF

Exercise 10: Implementing the MVC Pattern

Program :

```

class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

class StudentView {
    void display(String name) {
        System.out.println("Student Name: " + name);
    }
}

class StudentController {
    Student model;
    StudentView view;

    StudentController(Student m, StudentView v) {
        model = m;
        view = v;
    }

    void setName(String name) {
        model.name = name;
    }

    void updateView() {
        view.display(model.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student model = new Student("Bharath");
        StudentView view = new StudentView();
    }
}

```

```

        StudentController controller =
            new StudentController(model, view);

        controller.updateView();

        controller.setName("Venkat Bharath");

        controller.updateView();
    }
}

```

Output :

Student Name: Bharath
 Student Name: Venkat Bharath

Exercise 11: Implementing Dependency Injection

Program :

```

interface CustomerRepository {
    String findCustomerById(int id);
}

class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer ID: " + id;
    }
}

class CustomerService {
    private CustomerRepository repository;

    CustomerService(CustomerRepository repository) {
        this.repository = repository; // Dependency Injection
    }

    void getCustomer(int id) {
        System.out.println(repository.findCustomerById(id));
    }
}

public class Main {
    public static void main(String[] args) {

        CustomerRepository repo = new CustomerRepositoryImpl();
        CustomerService service = new CustomerService(repo);

        service.getCustomer(101);
    }
}

```

Output :

Customer ID: 101